

LAB 06 ASSIGNMENT

Performance Comparison of DFS-Backtracking and Local Search Using Heuristics

Application: Map Coloring Problem

1. Objective

The objective of this experiment is to model the **Map Coloring Problem** as a **Constraint Satisfaction Problem (CSP)** and experimentally compare the performance of two CSP-solving techniques:

1. **DFS-Backtracking with Heuristics**
2. **Local Search with Min-Conflicts**

The experiment aims to understand how the **number of variables, number of constraints, and problem size** affect the computation time and performance of both algorithms.

2. Problem Statement

Consider a map consisting of several regions.

Each region must be assigned **one color** such that **no two neighboring regions have the same color**.

The map is represented as a graph where:

- Each **region** represents a CSP variable.
- Each **color** represents a value in the variable's domain.
- An **edge between two regions** represents a constraint.
- Two connected regions must have **different colors**.

The available colors are:

Colors = {Red, Green, Blue}

Example

Consider a map with five regions:

Variables:

{A, B, C, D, E}

Domain:

{Red, Green, Blue}

Constraints:

A $\neq$ B

A $\neq$ C

A $\neq$ D

B $\neq$ C

$B \neq D$
 $C \neq D$
 $D \neq E$

Your task is to develop two CSP solvers that assign colors to all regions while satisfying all constraints:

- **DFS-Backtracking with MRV, LCV, and Forward Checking**
- **Local Search with Min-Conflicts**

You must experimentally compare the performance of both approaches.

3. CSP Model

3.1 Variables

Each region of the map is considered a CSP variable.

For example:

If the map contains 5 regions:

Variables = {A, B, C, D, E}

Therefore:

Number of Variables = Number of Regions

3.2 Domain

Each region can be assigned one of the following three colors:

Domain = {Red, Green, Blue}

For example:

$A \rightarrow \{\text{Red, Green, Blue}\}$

$B \rightarrow \{\text{Red, Green, Blue}\}$

$C \rightarrow \{\text{Red, Green, Blue}\}$

$D \rightarrow \{\text{Red, Green, Blue}\}$

$E \rightarrow \{\text{Red, Green, Blue}\}$

Each region must finally be assigned **exactly one color**.

3.3 Constraints

A constraint exists between two regions if they are connected by an edge.

For every edge:

(A, B)

the constraint is:

$A \neq B$

This means that neighboring regions cannot have the same color.

Important Rule

For this experiment:

Number of Constraints = Number of Edges

Each edge must be counted **only once**.

For example:

(A, B) and (B, A) represent the same constraint and must not be counted twice.

4. Example of a Valid Coloring

Consider the following assignments:

Region Color

A Red

B Green

C Blue

D Red

E Green

The program must check every constraint.

For example:

$A \neq B \rightarrow \text{Red} \neq \text{Green} \rightarrow \checkmark$

$A \neq C \rightarrow \text{Red} \neq \text{Blue} \rightarrow \checkmark$

$A \neq D \rightarrow \text{Red} \neq \text{Red} \rightarrow \times$

Therefore, this assignment is **not valid**.

The program must continue searching until it finds an assignment where **all constraints are satisfied**.

5. Part A — DFS-Backtracking with Heuristics

Implement a **DFS-Backtracking CSP solver** using the following heuristics.

5.1 Minimum Remaining Values (MRV)

Select the unassigned variable having the **smallest number of currently legal values**.

Example

A → {Red, Green}
B → {Red}
C → {Red, Green, Blue}
D → {Green, Blue}

MRV selects:

B

because B has only one remaining legal value.

Purpose

MRV attempts to assign the **most constrained variable first**, reducing unnecessary search.

5.2 Least Constraining Value (LCV)

When selecting a value for a variable, choose the value that **eliminates the fewest choices from neighboring unassigned variables**.

Example

For variable A:

Color **Values eliminated from neighbors**

Red 4

Green 2

Blue 1

LCV selects:

Blue

because it leaves the largest number of choices for the remaining variables.

5.3 Forward Checking

After assigning a color to a variable:

1. Check its unassigned neighbors.
2. Remove the assigned color from their domains.
3. If a neighbor's domain becomes empty, the assignment is invalid.
4. Backtrack and try another value.

Example

If:

A → Red

and A is connected to B and C:

Before:

$B \rightarrow \{\text{Red, Green, Blue}\}$

$C \rightarrow \{\text{Red, Green, Blue}\}$

After Forward Checking:

$B \rightarrow \{\text{Green, Blue}\}$

$C \rightarrow \{\text{Green, Blue}\}$

5.4 DFS-Backtracking Requirements

The program must record:

- Number of variables
 - Number of constraints
 - Number of assignments attempted
 - Number of backtracks
 - Execution time
 - Whether a valid solution was found
 - Final coloring
-

6. Part B — Local Search with Min-Conflicts

Implement a **Local Search CSP solver** using the **Min-Conflicts heuristic**.

The algorithm should perform the following steps:

Step 1 — Generate Initial Assignment

Generate a complete coloring randomly.

Example:

$A \rightarrow \text{Red}$

$B \rightarrow \text{Red}$

$C \rightarrow \text{Blue}$

$D \rightarrow \text{Green}$

$E \rightarrow \text{Red}$

Step 2 — Identify Conflicts

Check all constraints and identify the regions involved in violations.

For example:

A = Red

B = Red

If A and B are connected:

A $\neq$ B is violated.

Step 3 — Select a Conflicted Variable

Select one of the variables involved in a constraint violation.

Step 4 — Try Alternative Values

Try all possible colors for the selected variable.

For example:

Color for B Number of Conflicts

Red 2

Green 0

Blue 1

Select:

B $\rightarrow$ Green

because it produces the minimum number of conflicts.

Step 5 — Repeat

Continue the process until:

- A valid coloring is found, or
- The maximum number of iterations is reached.

Use:

Maximum Iterations = 10,000

If no valid solution is found within 10,000 iterations, report:

Solution Found: No

6.1 Local Search Requirements

The program must record:

- Number of variables

- Number of constraints
- Number of iterations
- Number of conflicts
- Execution time
- Whether a valid solution was found
- Final coloring

7. Part C — Vary the Problem Size

Run both algorithms on maps with increasing numbers of variables.

Use at least the following problem sizes:

Test Case Number of Variables

1	3
2	5
3	7
4	10
5	15
6	20
7	30
8	40
9	50

The number of colors must remain fixed for all experiments:

Colors = {Red, Green, Blue}

8. Graph Generation

For each problem size:

1. Generate the required number of regions.
2. Generate edges between regions.
3. Count the edges.
4. Treat each edge as one constraint.
5. Ensure that the generated graph is **3-colorable**.

6. Use the **same graph instance for both algorithms**.

Example

For 5 variables:

Variables:

A, B, C, D, E

Generated edges:

A-B

A-C

B-C

B-D

C-E

D-E

Therefore:

Number of Variables = 5

Number of Constraints = 6

Both algorithms must solve this **same graph**.

9. Important Requirement for Graph Generation

The generated graph must have at least one valid coloring using:

{Red, Green, Blue}

Do not generate graphs that require more than three colors.

The program should therefore ensure:

A valid 3-coloring exists before performing the performance experiment.

10. Part D — Performance Measurement

For every test case, measure the execution time of:

Algorithm 1

DFS-Backtracking + MRV + LCV + Forward Checking

Algorithm 2

Local Search + Min-Conflicts

For reliable results, run **each algorithm 10 times for every problem size**.

Therefore, for each test case:

10 DFS runs + 10 Local Search runs

Use Python's:
`time.perf_counter()`
for measuring execution time.

10.1 Average Execution Time

Calculate the average execution time using:

Average Time = $(T_1 + T_2 + \dots + T_{10}) / 10$

Report the average time in **milliseconds (ms)**.

11. Part E — Results Table

Prepare a table in the following format:

Variables Constraints		DFS Avg. Time (ms)	Local Search Avg. Time (ms)	Avg. Backtracks	Avg. Iterations	DFS Solution	Local Search Solution
3	—	—	—	—	—	—	—
5	—	—	—	—	—	—	—
7	—	—	—	—	—	—	—
10	—	—	—	—	—	—	—
15	—	—	—	—	—	—	—
20	—	—	—	—	—	—	—
30	—	—	—	—	—	—	—
40	—	—	—	—	—	—	—
50	—	—	—	—	—	—	—

Fill the table using the **actual values obtained from your program**.

Do not use theoretical or assumed execution times.

12. Part F — Plot the Results

Generate the following graphs.

Graph 1 — Number of Variables vs Computation Time

X-axis: Number of Variables

Y-axis: Average Computation Time (ms)

Plot two curves:

- DFS-Backtracking
 - Local Search
-

Graph 2 — Number of Constraints vs Computation Time

X-axis: Number of Constraints

Y-axis: Average Computation Time (ms)

Plot two curves:

- DFS-Backtracking
 - Local Search
-

13. Part G — Analysis and Discussion

Based on the experimental results and graphs, answer the following questions.

1.

Which algorithm performs better for small CSPs?

2.

Which algorithm performs better as the number of variables increases?

3.

How does increasing the number of constraints affect DFS-Backtracking?

4.

How does increasing the number of constraints affect Local Search?

5.

How does the number of backtracks change as the problem size increases?

6.

How does the number of iterations in Local Search change as the problem size increases?

7.

How does the **MRV heuristic** improve DFS-Backtracking?

8.

What is the purpose of the **LCV heuristic**?

9.

How does **Forward Checking** reduce unnecessary search?

10.

How does the **Min-Conflicts heuristic** help Local Search?

11.

Is Local Search guaranteed to find a valid solution? Explain.

12.

Is DFS-Backtracking guaranteed to find a solution if a solution exists? Explain.

13.

At approximately what problem size does one approach begin to outperform the other in your experiment?

14.

Based on your experimental results, which algorithm would you recommend for:

a. Small CSPs?

b. Large CSPs?

Give reasons.

15.

Explain the relationship between:

- Number of variables
- Number of constraints
- Computation time

14. Expected Output

For every test case, display output similar to:

=====

Test Case: 4

=====

Number of Variables: 10

Number of Constraints: XX

Number of Colors: 3

DFS-BACKTRACKING

Solution Found: Yes

Average Time: XX.XX ms

Average Assignments: XX

Average Backtracks: XX

LOCAL SEARCH

Solution Found: Yes

Average Time: XX.XX ms

Average Iterations: XX

Average Conflicts: XX

Final Coloring:

A → Red

B → Green

C → Blue

D → Red

...

15. Final Submission

Students must submit the following:

15.1 Source Code

Complete implementation of:

- CSP representation
- Graph generation
- Domain representation
- Constraint checking
- DFS-Backtracking
- MRV

- LCV
 - Forward Checking
 - Local Search
 - Min-Conflicts
 - Execution-time measurement
-

15.2 Experimental Results

Submit the completed results table for all problem sizes.

15.3 Graphs

Include:

1. **Number of Variables vs Computation Time**
 2. **Number of Constraints vs Computation Time**
-

15.4 Analysis

Provide answers to all questions in **Part G**.

15.5 Conclusion

Write a brief conclusion explaining:

- Which algorithm performed better
 - How performance changed as the problem size increased
 - How the number of constraints affected computation time
 - Which algorithm you would recommend for larger CSPs
 - Why the selected algorithm performed better
-

16. Important Instructions

1. Both algorithms must be tested on the **same graph instance** for each test case.
2. Use exactly **three colors**:
{Red, Green, Blue}
3. Every edge represents **one constraint**.
4. Count each edge only once.

5. The generated graph must be **3-colorable**.
 6. Run each algorithm **10 times** for every test case.
 7. Use actual execution times obtained from the program.
 8. Do not manually enter or assume execution times.
 9. Use `time.perf_counter()` for timing.
 10. Use a maximum of **10,000 iterations** for Local Search.
 11. Record whether a valid solution was found.
 12. Clearly report the number of variables and constraints.
 13. Clearly label the axes and legends of all graphs.
 14. Both algorithms must use the **same colors, variables, constraints, and graph** for a fair comparison.
 15. The final results must be based on **actual program execution**.
-

Main Goal of the Lab

Experimentally compare systematic search (DFS-Backtracking) and heuristic local search (Min-Conflicts) and determine how their computational performance changes as the number of variables and constraints increases.

CSP Summary

Variables → Regions

Domain → {Red, Green, Blue}

Constraints → Edges

Constraint Rule → Adjacent regions must have different colors

Goal → Find a valid coloring satisfying all constraints